

$$8.1.4 \quad F(n) = \min_{j: n \geq d_j} \{F(n-d_j)\} + 1$$

$$F[0] = 0$$

$$F[1] = \min\{F[0]\} + 1 = 1$$

$$F[2] = \min\{F[1]\} + 1 = 2$$

$$F[3] = \min\{F[2], F[0]\} + 1 = \min\{2, 0\} + 1 = 1$$

$$F[4] = \min\{F[3], F[1]\} + 1 = 2$$

$$F[5] = \min\{F[4], F[2], F[0]\} + 1 = 1$$

$$F[6] = \min\{F[5], F[3], F[1]\} + 1 = 2$$

$$F[7] = \min\{F[6], F[4], F[2]\} + 1 = 3$$

$$F[8] = \min\{F[7], F[5], F[3]\} + 1 = 2$$

$$F[9] = \min\{F[8], F[6], F[4]\} + 1 = 3$$

故最少硬币数 $F[9] = 3$, 硬币组合 $(1, 3, 5)$, $(3, 3, 3)$

8.1.5 修改: ① 初始化: 障碍单元格标记为不可达

② 只对上方或左侧可达的单元格进行更新; 若均不能达, 则当前单元格也不能达.

③ 增加路径计数矩阵 $count[i, j]$, 记录到达 (i, j) 且硬币数最大的路径数.

求解: 最大硬币数 $F[5, 6] = 4$, $count[5, 6] = 12$.

8.2.5

$$F(w) = \max_{j: w_j \leq w} [F(w - w_j) + v_j]$$

初始条件 $F[0] = 0$, 若所有 $w_j > w$, 则 $F(w) = 0$

时间复杂度: 由于算法对 W 和 n 双重循环, 故时间复杂度 $O(nW)$

8.3.1

A: 0.1 B: 0.2 C: 0.4 D: 0.3

$$C[1,2]: k=1, C[1,0] + C[2,2] + \sum_{s=1}^2 p_s = 0.5$$

$$k=2, C[1,1] + C[3,2] + \sum_{s=1}^2 p_s = 0.4$$

$$\text{则 } C[1,2] = 0.4$$

$$C[2,3]: k=2, C[2,1] + C[3,3] + \sum_{s=2}^3 p_s = 1$$

$$k=3, C[2,2] + C[4,3] + \sum_{s=2}^3 p_s = 0.8$$

$$\text{则 } C[2,3] = 0.8$$

$$C[3,4]: k=3, C[3,2] + C[4,4] + \sum_{s=3}^4 p_s = 1$$

$$k=4, C[3,3] + C[5,4] + \sum_{s=3}^4 p_s = 1.1$$

$$\text{则 } C[3,4] = 1$$

$$C[1,3]: k=1, C[1,0] + C[2,3] + \sum_{s=1}^3 p_s = 1.5$$

$$k=2, C[1,1] + C[3,3] + \sum_{s=1}^3 p_s = 1.2$$

$$k=3, C[1,2] + C[4,3] + \sum_{s=1}^3 p_s = 1.1$$

$$\text{则 } C[1,3] = 1.1$$

$$C[2,4]: k=2, C[2,1]+C[3,4]+\sum_{s=2}^4 ps=1.9$$

$$k=3, C[2,2]+C[4,4]+\sum_{s=2}^4 ps=1.4$$

$$k=4, C[2,3]+C[5,4]+\sum_{s=2}^4 ps=1.7$$

$$\text{则 } C[2,4]=1.4$$

$$C[1,4]: k=1, C[1,0]+C[2,4]+\sum_{s=1}^4 ps=2.4$$

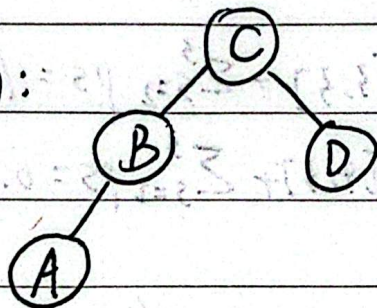
$$k=2, C[1,1]+C[3,4]+\sum_{s=1}^4 ps=2.1$$

$$k=3, C[1,2]+C[4,4]+\sum_{s=1}^4 ps=1.7$$

$$k=4, C[1,3]+C[5,4]+\sum_{s=1}^4 ps=2.1$$

$$\text{则 } C[1,4]=1.7$$

最终树为:



8.3.3 伪代码: Algorithm Treebest(i, j)

// 输入: 整数 i, j , 表示第 i 到第 j 个关键字

// 输出: 按先序遍历输出最优二叉索引树的节点索引, $R[1..n, 1..n]$ 为动态规划的结果表

if $i \leq j$

$k \leftarrow R[i, j]$

print(k)

Treebest($i, k-1$)

Treebest($k+1, j$)

8.4.1

$$R(0) = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$k=1, R(1) = R(0) = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$k=2, R(2) = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$k=3, R(3) = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$k=4, \text{无更新}, T = R(4) = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$8.4.7 \quad D(0) = \begin{pmatrix} 0 & 2 & \infty & 1 & 8 \\ 6 & 0 & 3 & 2 & \infty \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & \infty & \infty & \infty & 0 \end{pmatrix}$$

$$D(1) = \begin{pmatrix} 0 & 2 & \infty & 1 & 8 \\ 6 & 0 & 3 & 2 & 14 \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & 5 & \infty & 4 & 0 \end{pmatrix}$$

$$D(2) = \begin{pmatrix} 0 & 2 & 5 & 1 & 8 \\ 6 & 0 & 3 & 2 & 14 \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & 5 & 8 & 4 & 0 \end{pmatrix}$$

$$D(3) = \begin{pmatrix} 0 & 2 & 5 & 1 & 8 \\ 6 & 0 & 3 & 2 & 14 \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & 5 & 8 & 4 & 0 \end{pmatrix}$$

$$D(4) = \begin{pmatrix} 0 & 2 & 3 & 1 & 4 \\ 6 & 0 & 3 & 2 & 5 \\ \infty & \infty & 0 & 4 & 7 \\ \infty & \infty & 2 & 0 & 3 \\ 3 & 5 & 6 & 4 & 0 \end{pmatrix}$$

$$D(5) = \begin{pmatrix} 0 & 2 & 3 & 1 & 4 \\ 6 & 0 & 3 & 2 & 5 \\ 10 & 12 & 0 & 4 & 7 \\ 6 & 8 & 2 & 0 & 3 \\ 3 & 5 & 6 & 4 & 0 \end{pmatrix} = D$$

附加题：给定正整数 n ，设计高效算法，找出最少个数的完全平方数（如 1, 4, 9, 16, ...），使这些完全平方数的和等于 n 。

(1) 简要说明设计算法的策略或思想，并用伪代码描述算法。

(2) 分析所设计算法的时间复杂度。

(1) 采用动态规划的方法，设 $F(i)$ 表示整数 i 最少需几个完全平方数相加得到， $F(i) = \min\{F(i - j*j) + 1\}$

对以下伪代码的说明：对每个 i ，尝试减 $j*j$ 完全平方数，剩余部分为 $i - j*j$ 。 $F(i - j*j)$ 已是最优解，加上当前平方数即可得一种方案，取所有方案中的最小值。

伪代码:

Algorithm MinnumSquare (n)

// 输入: 正整数 n

// 输出: 和为 n 的最少完全平方数个数.

$F(0) \leftarrow 0$

for $i \leftarrow 1$ to n do

$F(i) \leftarrow i$

for $i \leftarrow 1$ to n do

$j \leftarrow 1$

while $j * j \leq i$ do

$F(i) \leftarrow \min(F(i), F(i - j * j) + 1)$

$j \leftarrow j + 1$

return $F(n)$

(2) 外层循环执行 n 次, 每个 i 内层循环最多执行 $\sqrt{i}$ 次.

故时间复杂度为 $O(n\sqrt{n})$